

Entropy Landscapes and Kinetics of Classical Sorting Algorithms

Punyajit Mishra

Abstract—Sorting algorithms are usually compared by asymptotic complexity, which says little about how a particular run moves through the space of array arrangements. We treat a permutation of n distinct keys as a microstate and its inversion count m as a macrostate, with exact multiplicities $\Omega_n(m)$ from the Mahonian distribution, entropy $S(m) = \ln \Omega_n(m)$, and a normalised disorder energy $E = m/m_{\max}$. Instrumented Bubble, Insertion, Merge, Heap and Quick Sort record trajectories (E_t, S_t) together with comparisons and writes; a cost $W = C + M$ defines the disorder removed per unit cost, $R = (E_0 - E_f)/W$. We prove exactly how each algorithm’s steps change m : Bubble and Quick Sort never increase it, Insertion and Merge Sort remove it in data-determined batches, and every sift-down swap in Heap Sort increases it. In the $n = 100$ trajectory experiment over 30 uniform-random inputs, Heap Sort raises E in 83.0% of recorded steps, from 0.50 to a mean peak of 0.79, and traverses 3.28 times its net entropy change, against 1.002 for the other four. Reading an effective inverse temperature $\beta_{\text{eff}} = dS/dE$ off the exact landscape shows this excursion is not merely larger but qualitatively different: the four monotone algorithms have mean most-negative $\beta_{\text{eff}} \approx -3.9$, while Heap Sort’s mean most-negative value is $\beta_{\text{eff}} \approx -343$, nearly two orders of magnitude larger in magnitude. Cost rankings depend on input shape and size: last-pivot Quick Sort is cheapest on uniform-random input but third on nearly-sorted and on reverse-sorted input, and Insertion Sort overtakes Merge Sort in cost at $n \approx 23$. Since $R = E_0/W$ for complete sorts, R normalises across input distributions but adds no ranking information beyond W ; the entropy-path and inverse-temperature statistics do. The framework is abstract and measures no physical energy.

Index Terms—sorting algorithms, inversion count, permutation entropy, presortedness, computational kinetics, algorithmic thermodynamics

I. INTRODUCTION

ALGORITHMS are usually compared by how their resource use scales with input size [1]. Two runs with the same cost can nevertheless pass through very different sequences of intermediate arrangements, and the same algorithm can behave very differently on inputs of different shape. Adaptive sorting studies the shape dependence through measures of presortedness such as the number of inversions and the number of runs [33], [34]; entropy has been used to analyse comparison sorting [4], [6], [7] and to design entropy-sensitive algorithms [5]. This paper adds a statistical-mechanical description of the *path* an algorithm takes: how its disorder and the entropy of its current macrostate evolve while it runs.

The vocabulary is borrowed from the thermodynamics of computation, in which logically irreversible operations carry

a minimum dissipation cost [9], computation can in principle be made reversible [10], program properties can be treated as observables of a Gibbs ensemble [11], and statistical-physics methods explain phase transitions in computational hardness [12]. We make no physical-energy claim. A separate empirical literature measures the energy that sorting code actually draws from real hardware, for instance in energy-constrained comparison models [21] and on smartphones [22]; our framework is complementary because it instruments the abstract state space rather than a device.

Contributions. (i) We compute the entropy landscape exactly with big-integer arithmetic and quantify the error of the Gaussian approximation. (ii) We prove how each algorithm’s recorded steps change the inversion count (Section IV). (iii) We introduce trajectory statistics, including an entropy path length that separates Heap Sort from the four algorithms whose trajectories are monotone. (iv) We report paired experiments with 1000 trials per input distribution at $n = 100$, a size sweep to $n = 2000$, and pivot and run-adaptive variants. (v) We show that the disorder reduction rate R equals E_0/W and therefore ranks algorithms exactly as W does within a fixed distribution.

II. BACKGROUND

A. Permutations and inversions

An inversion of a permutation π of n distinct keys is a pair of positions $i < j$ with $\pi_i > \pi_j$, and

$$m(\pi) = |\{(i, j) : i < j, \pi_i > \pi_j\}|, \quad (1)$$

so $m \in \{0, \dots, m_{\max}\}$ with $m_{\max} = \binom{n}{2}$. The sorted permutation has $m = 0$ and the reverse-sorted one has $m = m_{\max}$. The statistic is a classical object of sorting analysis [1] and of enumerative combinatorics [14], and the combinatorics of permutations more broadly is surveyed in [15].

B. Multiplicity and exact entropy

Let $\Omega_n(m)$ be the number of permutations with m inversions. These Mahonian numbers [13] satisfy

$$\sum_{m=0}^{m_{\max}} \Omega_n(m) q^m = \prod_{k=1}^n \frac{1 - q^k}{1 - q}, \quad (2)$$

and the counts of permutations by inversions have been studied in their own right [16]. Ordering and mixing in a physical system of diffusing particles is analysed in [17]. We define the macrostate entropy in reduced units as

$$S(m) = \ln \Omega_n(m), \quad \Omega_n(0) = \Omega_n(m_{\max}) = 1. \quad (3)$$

$\Omega_n(m)$ is symmetric about $m_{\max}/2$ and, for large n , approximately Gaussian with mean $\mu = n(n-1)/4$ and variance $\sigma^2 = n(n-1)(2n+5)/72$. We do not use the Gaussian form: multiplying out (2) with arbitrary-precision integers is inexpensive (the $n = 100$ table is built in under 0.1 s), and Section VI-A shows that the Gaussian approximation is badly wrong in the tails that nearly-sorted inputs occupy. Our implementation is validated against brute-force enumeration for $n = 5, 6, 7$ and by $\sum_m \Omega_n(m) = n!$ and symmetry at $n = 100$.

C. Combinatorial and Shannon entropy

$S(m)$ is a Boltzmann-type entropy: it counts the configurations compatible with a macrostate. It is distinct from the Shannon entropy of a random variable, for which Cover and Thomas give the standard development, including mutual information and the asymptotic equipartition property [2]. Gray's text treats the same material with more emphasis on the entropy rates of random processes [3]. The two are related by a change of base, $S(m)/\ln 2$ being the number of bits needed to single out one of the $\Omega_n(m)$ permutations in the macrostate.

D. Disorder energy and ensemble

The dimensionless disorder energy is $E(\pi) = m(\pi)/m_{\max} \in [0, 1]$. For an inverse-temperature parameter β it defines a canonical ensemble $p_\beta(\pi) \propto e^{-\beta E(\pi)}$ with partition function $Z(\beta) = \sum_m \Omega_n(m) e^{-\beta m/m_{\max}}$ and free energy $F = -\beta^{-1} \ln Z$. E is a chosen observable in the same sense as the observables of algorithmic thermodynamics [11]. The ensemble is defined for completeness; the results below use only E and S along trajectories.

E. Thermodynamics of computation

Landauer's bound [9] and Bennett's reversible computation [10] anchor the field. The minimal work cost of information processing has been formulated for general tasks in an information-theoretic setting [18]; historical accounts trace how these ideas developed at IBM [20]; overviews mark sixty years of Landauer's principle [25], and recent work extends the thermodynamics of computation to irreversible transitions and stochastic computation times [28], and [31] discusses the principle together with the thermodynamics of computation. A physical lower bound for a sorting run would require knowing which bits a specific machine erases; our simulation does not, so these results are background only.

F. Entropy in sorting theory

The decision-tree argument shows that any comparison sort needs at least $\log_2 n!$ comparisons in the worst case [1], [8]. Kahn and Kim relate the number of comparisons needed when partial order information is available to the entropy of the incomparability graph [4]. Schellekens proves an entropy conservation law for comparison-based algorithms [6], List *et al.* tie the expected comparisons of randomised Quick Sort to the entropy of the pivot source [7], and Gila *et al.* define a

run-based entropy for entropy-sensitive in-place sorting [5]. None of these tracks a scalar macrostate along a specific run, which is what we do.

III. FRAMEWORK

A. States and recording

A state is recorded only when the full working array is a permutation of the input. Bubble, Quick and Heap Sort record after every swap. Insertion Sort records after each completed insertion, and Merge Sort after each completed merge; buffers containing duplicated keys are never recorded. Comparisons and writes are counted at their true frequency: a swap costs three writes, each shift or copy one write.

B. Cost and disorder reduction rate

The abstract cost is $W = C + M$, comparisons plus writes. Counting writes separately from comparisons follows the asymmetric read/write cost view of sorting [19]; here the two weights are equal. The disorder reduction rate is

$$R = \frac{E_0 - E_f}{W}. \quad (4)$$

Proposition 1. *For an algorithm that sorts completely ($E_f = 0$), $R = E_0/W$. Hence, on any fixed input, ordering algorithms by decreasing R is ordering them by increasing W . The statement is a within-input ranking result; R does not make costs from different input distributions directly comparable without specifying the starting disorder.*

This is immediate from (4). It means R is a normalisation that makes costs comparable across distributions with different starting disorder; it is not an independent efficiency measure, and we never treat it as one.

C. Trajectory statistics

For a recorded sequence $m_0, m_1, \dots, m_T$ we report the number of steps T , the fraction of steps with $m_{t+1} > m_t$, the peak $\max_t E_t$, the largest single-step drop as a fraction of m_0 , and the entropy path length

$$\Lambda = \frac{\sum_{t=0}^{T-1} |S(m_{t+1}) - S(m_t)|}{S(m_0)}. \quad (5)$$

For $S(m_0) > 0$, every complete sort has the same net change $S(m_0) \rightarrow 0$, so Λ measures the total entropy variation relative to the initial entropy. $\Lambda = 1$ when S decreases monotonically, while $\Lambda > 1$ records an entropy detour, which can arise either from reversals in S or from crossing the entropy peak even when m itself changes monotonically. The peak and the fraction of increasing steps both characterize upward excursions in the E coordinate, but they are not equivalent statistics.

We also read an effective inverse temperature off the exact landscape,

$$\beta_{\text{eff}}(m) = \left. \frac{dS}{dE} \right|_m \approx m_{\max} \frac{S(m+1) - S(m-1)}{2}, \quad (6)$$

a central difference defined for $0 < m < m_{\max}$. The endpoint states $m = 0$ and $m = m_{\max}$ therefore have no β_{eff} value under this definition and are excluded from β_{eff} -based trajectory statistics. In the canonical ensemble of Section II-D, $\beta_{\text{eff}}(m)$ is the local inverse-temperature value associated with macrostate m : in the continuous approximation, the modal condition is $\beta_{\text{eff}} = \beta$. Thus it converts a raw inversion count into the inverse-temperature scale of the ensemble in which that count is typical. $\beta_{\text{eff}} > 0$ below the entropy peak and $\beta_{\text{eff}} < 0$ above it; for odd $m_{\max}$, the two central macrostates straddle $\beta_{\text{eff}} = 0$. Because the disorder energy is bounded, negative β_{eff} values are mathematically allowed and correspond to the high-disorder side of the landscape, not to a physically colder state. We use it to measure how far a trajectory strays from the high-entropy centre of the landscape (Section VI-G).

IV. STRUCTURAL PROPERTIES OF THE TRAJECTORIES

Proposition 2 (Swap lemma). *Exchanging positions $i < j$ with $\pi_i > \pi_j$ (an inverted pair) reduces the inversion count by exactly $1 + 2c$, where c is the number of positions k with $i < k < j$ and $\pi_j < \pi_k < \pi_i$. Exchanging a non-inverted pair $i < j$, $\pi_i < \pi_j$, increases it by exactly $1 + 2c'$ with $c' = |\{k : i < k < j, \pi_i < \pi_k < \pi_j\}|$.*

Proof. Only pairs involving i or j can change. The pair (i, j) itself flips. For an intermediate k with value below both or above both of π_i, π_j , the two pairs (i, k) and (k, j) contribute the same number of inversions before and after. If π_k lies strictly between the two values, the two pairs contribute two inversions when $\pi_i > \pi_j$ and none after the swap, and the reverse for the non-inverted case. Pairs with an endpoint outside $[i, j]$ are unchanged. The second statement is the first read backwards. $\square$

Proposition 3 (Bubble Sort). *Every swap lowers m by exactly one, so the recorded trajectory descends linearly in exactly m_0 steps for every input.*

Proof. Adjacent positions have no intermediate k , so Proposition 2 gives $c = 0$ and Bubble Sort only swaps inverted pairs. $\square$

Proposition 4 (Quick Sort with Lomuto partition). *Every recorded nontrivial swap exchanges an inverted pair, so m never increases and each nonzero drop is odd.*

Proof. Within a partition the region $\pi_i, \dots, \pi_{j-1}$ holds keys larger than the pivot. Every recorded nontrivial swap of π_i with $\pi_j \leq \text{pivot}$ therefore exchanges an inverted pair ($i < j$, $\pi_i > \pi_j$), and the nontrivial final swap of π_i with the pivot does likewise. Self-swaps are not recorded as swaps. Apply Proposition 2. $\square$

Individual swaps can nevertheless be long: the drop grows with the swap distance through c , unlike Bubble Sort's unit steps.

Proposition 5 (Insertion Sort). *The recorded drop at the insertion of π_i equals the number of larger keys in the sorted prefix, and the drops sum to m_0 . There are $n - 1$ recorded steps, and a step is zero when the key is already in place.*

Proof. Inserting a key into a sorted prefix removes exactly the inversions it forms with that prefix and no others. $\square$

Proposition 6 (Merge Sort). *When the merge with split point s completes, it removes exactly the cross inversions between the two halves, and every inversion (i, j) is removed at the unique merge whose split separates i and j . There are $n - 1$ recorded steps, and on a uniformly random input the last merge removes on average a fraction $\frac{(n/2)^2/2}{n(n-1)/4} \approx \frac{1}{2}$ of the initial inversions.*

Proof. By induction both halves are sorted when they are merged, so all their internal inversions are already zero; the merge sorts the union and leaves inversions with positions outside the block untouched. Each pair of positions is separated by exactly one recursion node, and the expected number of cross inversions between two random halves of size $n/2$ is $(n/2)^2/2$. $\square$

Proposition 7 (Heap Sort). *Every swap inside a sift-down exchanges a parent with a strictly larger child, that is, a non-inverted pair, and therefore increases m by $1 + 2c'$. Only the $n - 1$ root-last extraction swaps exchange an inverted pair and decrease m . If a run makes s recorded swaps, the fraction of steps at which m increases is exactly $1 - (n - 1)/s$.*

Proof. A max-heap sift-down moves the smaller parent below the larger child, which has a larger index. The root is the maximum of the active heap, so the extraction swap exchanges it with a smaller key at a larger index. Apply Proposition 2. $\square$

Heap Sort therefore creates positional disorder while it creates heap order. This is not a defect: the inversion count measures linear order, and the heap invariant is a different kind of order that this scalar does not see.

V. EXPERIMENTAL DESIGN

A. Implementations and validation

The instrumented implementations (Bubble, Insertion, Merge, Heap, and Quick Sort with a last-element Lomuto pivot) record states as in Section III and check on every state that the array is a permutation of the input. Large experiments use count-only versions with identical accounting; these agree with the instrumented versions on all (C, M) values for 20 test arrays of size 30 across four distributions and five algorithms, and every output was verified sorted. Three variants are added: Quick Sort with median-of-three pivot (three comparisons charged per partition) and with a uniformly random pivot, both swapped into the last position, and a natural merge sort that finds maximal ascending runs with $n - 1$ comparisons and merges adjacent runs bottom-up.

B. Input distributions and design

Uniform-random: a uniformly random permutation. *Nearly-sorted*: the identity with $\lfloor 0.05n \rfloor$ random transpositions. *Reverse-sorted*: $(n - 1, \dots, 0)$. *Few-unique*: n draws from five integer values; entropy and R are undefined here, so only C , M , W are reported. Within a distribution all algorithms sort the same arrays. The main experiment uses $n = 100$ and 1000 trials per random distribution (one for the deterministic reverse

TABLE I
EXACT AND GAUSSIAN-APPROXIMATE ENTROPY $S(m)$ (NATS) AT
 $n = 100$.

E	m	Exact	Gaussian	Difference
0.50	2475	357.69	357.70	0.00
0.30	1485	339.30	340.31	1.02
0.10	495	256.94	288.16	31.22
0.06	303	216.96	274.01	57.05
0.05	248	200.94	269.72	68.79
0.01	50	91.81	253.38	161.57

input); trajectories use 30 uniform-random arrays; the size sweep uses $n \in \{10, 20, 50, 100, 200, 500\}$ with 100 trials and $n \in \{1000, 2000\}$ with 30; a fine sweep uses $n = 10, \dots, 60$ with 400 trials. Every random stream derives from a fixed NumPy SeedSequence (entropy 20260927) with distinct spawn keys, so results do not depend on execution order. We report means and standard deviations across trials; the standard error is the standard deviation over $\sqrt{N}$.

VI. RESULTS

A. Entropy landscape

For $n = 10$ the exact landscape has equal maxima at $m = 22$ and $m = 23$, both with $S = 12.432$ (the Gaussian form gives 12.460), and $S(17) = 11.997$. For $n = 100$ it peaks at $m = 2475$ with $S = 357.694$ nats, which is 98.3% of $\ln 100! = 363.739$. The modal macrostate has multiplicity $2^{516.0}$, compared with $2^{524.8}$ possible permutations in total. The mean initial disorder of uniform-random arrays is $\langle E_0 \rangle = 0.501 \pm 0.035$ (the exact standard deviation is 0.0339), of nearly-sorted arrays 0.061 ± 0.019 , and of reverse-sorted arrays 1. Table I shows why exactness matters: the Gaussian approximation is accurate at the centre but overstates S by 31 nats at $E = 0.10$, 69 nats at $E = 0.05$ and 162 nats at $E = 0.01$.

B. An illustrative trial ($n = 10$)

Table II shows all five algorithms on one uniform-random input, (7, 3, 0, 5, 1, 8, 2, 6, 4, 9), with $m_0 = 17$ ($E_0 = 0.378$). Bubble Sort descends by one per step; Insertion and Merge Sort show plateaus and batch drops; Quick Sort drops at every swap by an odd amount; Heap Sort makes an upward excursion from 17 to 36 ($E = 0.80$) before descending non-monotonically. A single trial is anecdotal: at $n = 10$ the mean costs over 100 trials rank Quick (53), Insertion (62), Merge (91), Bubble (110), Heap (119) (Table V).

C. Trajectory statistics ($n = 100$, uniform-random)

Fig. 1 plots the mean trajectories over 30 inputs and Table III summarises them. The statistics match Section IV: Bubble Sort takes 2497.7 steps ($\approx m_0$) with no increases; Insertion and Merge Sort take 99 steps; Merge Sort's largest single drop is 50.7% of m_0 (prediction $\approx 50.5\%$), whereas Quick Sort's largest is 4.0% and Bubble Sort's is one inversion. Heap Sort increases m in 83.0% of its 580.9 recorded steps

(the exact formula $1 - (n-1)/s$ gives 82.96%), reaches a mean peak of $E = 0.786$ from $E_0 = 0.50$, and has $\Lambda = 3.28 \pm 0.12$, against 1.002 for each of the other four. The other four algorithms have almost identical Λ (slightly above one because some m_0 values lie above the entropy peak) despite a factor of eight in cost; the entropy path reveals a qualitative distinction that the scalar cost W does not.

D. Cost at $n = 100$

Table IV gives W and R over 1000 paired trials. On uniform-random input the order by cost is Quick (1486), Merge (1886), Heap (2772), Insertion (5154), Bubble (12321). Merge Sort's cost is nearly data-independent (standard deviation 5.8, from comparisons only, since its writes are fixed at $M(100) = 1344$ by the recurrence $M(n) = M(\lceil n/2 \rceil) + M(\lfloor n/2 \rfloor) + 2n$). On nearly-sorted input the order among the five original algorithms changes to Insertion (801), Merge (1798), Quick (2654), Heap (2978), Bubble (5050), so last-pivot Quick Sort moves from first to third; its cost is 2.3 times that of the random-pivot and median-of-three variants (1159 and 1162), consistent with the entropy-of-pivot-source picture [7]. On reverse-sorted input the order is Merge (1660), Heap (2492), Quick (5100), Insertion (9999), Bubble (19800); Quick Sort's comparison count reaches its maximum $\binom{100}{2} = 4950$ while its writes are only 150. The ranking by mean R is identical to the ranking by mean W in every distribution, as Proposition 1 leads one to expect.

E. Scaling with n and crossovers

Table V and Fig. 2 show mean cost against n . For this implementation the expected Insertion Sort cost on random input is exactly $\mathbb{E}[W_{\text{ins}}] = n(n-1)/2 + 2(n-1) - (H_n - 1)$, since writes equal inversions plus $n-1$ and comparisons equal inversions plus $n-1$ minus the number of prefix minima ($\mathbb{E}[\text{prefix minima}] = H_n - 1$, where H_n is the n th harmonic number); it gives 5143.8 at $n = 100$ against 5153.9 ± 11.1 (standard error) measured. In the fine sweep Insertion Sort is already more expensive than Quick Sort at $n = 10$ (61.5 against 53.5), becomes more expensive than Merge Sort at $n = 23$ (292.7 against 289.3, 400 trials; it is cheaper at $n \leq 22$) and more expensive than Heap Sort between $n = 35$ and $n = 40$. The near-tie between Insertion and Quick Sort in a single small trial is therefore not typical. On nearly-sorted input last-pivot Quick Sort degrades relative to the pivot variants as n grows (103,462 against 52,797 for random pivot at $n = 2000$). On few-unique input it is cheapest at $n = 100$ (1607) but grows quadratically thereafter (106,092 at $n = 1000$ and 412,327 at $n = 2000$, a factor 3.9), because with $k = 5$ values a $\leq$ -pivot Lomuto partition leaves all keys equal to the pivot on one side.

F. Run structure and natural merge

The run-based entropy of ascending runs, with logarithms to base two, averages 5.51 bits for uniform-random arrays, 2.89 for nearly-sorted, 5.18 for few-unique and 6.64 for reverse-sorted (ascending runs only, so descending input is maximally fragmented). Natural merge sort reduces cost most where run

TABLE II

ONE PAIRED TRIAL AT $n = 10$, $m_0 = 17$. $W = C + M$; STEPS IS THE NUMBER OF RECORDED STATES AFTER THE FIRST; THE LAST COLUMN LISTS m_t .

Algorithm	C	M	W	Steps	Recorded m_t
Bubble	35	51	86	17	$17 \rightarrow 16 \rightarrow 15 \rightarrow 14 \rightarrow 13 \rightarrow 12 \rightarrow 11 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
Insertion	24	26	50	9	$17 \rightarrow 16 \rightarrow 14 \rightarrow 13 \rightarrow 10 \rightarrow 10 \rightarrow 6 \rightarrow 4 \rightarrow 0 \rightarrow 0$
Merge	23	68	91	9	$17 \rightarrow 16 \rightarrow 14 \rightarrow 13 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 8 \rightarrow 6 \rightarrow 0$
Heap	39	87	126	29	$17 \rightarrow 26 \rightarrow 27 \rightarrow 30 \rightarrow 35 \rightarrow 36 \rightarrow 21 \rightarrow 24 \rightarrow 25 \rightarrow 18 \rightarrow 19 \rightarrow 20 \rightarrow 21 \rightarrow 16 \rightarrow 17 \rightarrow 18 \rightarrow 9 \rightarrow 10 \rightarrow 13 \rightarrow 4 \rightarrow 5 \rightarrow 10 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 0 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow 0$
Quick (last pivot)	25	33	58	11	$17 \rightarrow 16 \rightarrow 15 \rightarrow 12 \rightarrow 11 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

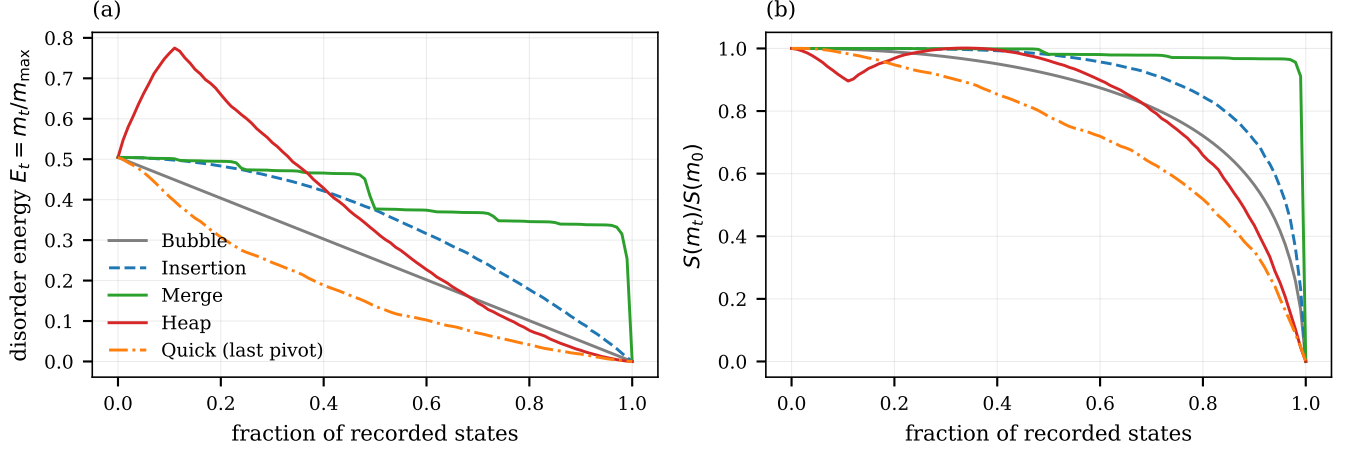
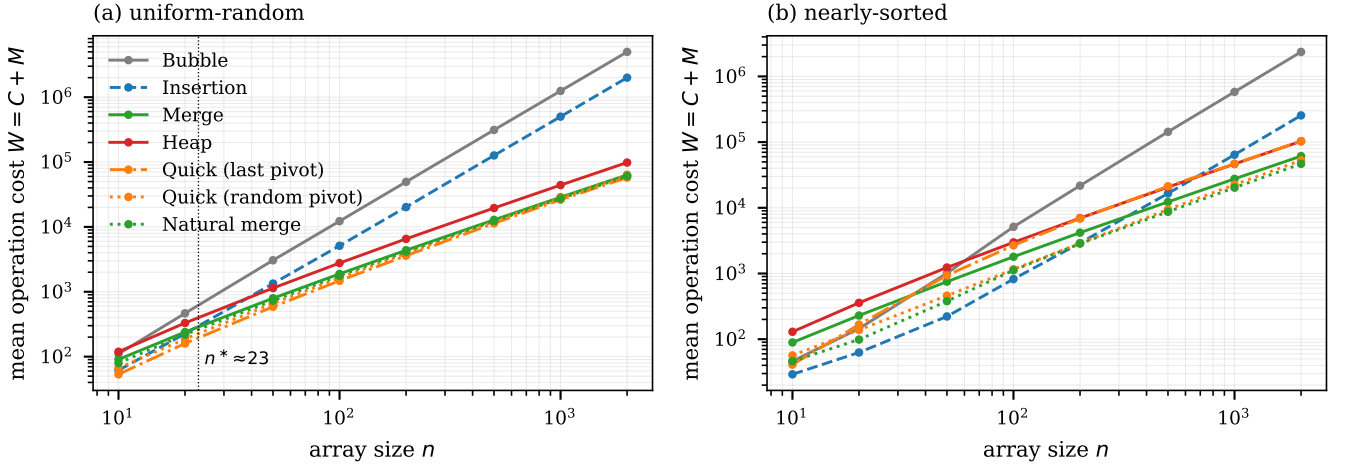
Fig. 1. Mean trajectories over 30 uniform-random inputs at $n = 100$, against the fraction of recorded states. (a) Disorder energy E_t . (b) Exact entropy $S(m_t)$ relative to its initial value. Heap Sort is the only algorithm whose E rises and whose entropy first falls.Fig. 2. Mean cost $W = C + M$ against n on log-log axes: (a) uniform-random, with the Insertion/Merge crossover marked; (b) nearly-sorted.

TABLE III

TRAJECTORY STATISTICS AT $n = 100$ (30 UNIFORM-RANDOM INPUTS).

Algorithm	Steps	Incr. (%)	Peak E	Max drop ($\%m_0$)	Δ
Bubble	2497.7	0.0	0.505	0.04	1.002
Insertion	99.0	0.0	0.505	3.55	1.002
Merge	99.0	0.0	0.505	50.72	1.002
Heap	580.9	83.0	0.786	7.36	3.284
Quick (last pivot)	282.6	0.0	0.505	3.96	1.002

entropy is low: relative to standard Merge Sort it is 6% cheaper on uniform-random input, 38% cheaper on nearly-sorted input (1119 against 1798, $R = 0.543 \times 10^{-4}$ against 0.338×10^{-4}), and 8% more expensive on reverse-sorted input, where its ascending-run detection finds nothing to exploit. At $n = 100$ Insertion Sort is still cheaper on nearly-sorted input (801), but natural merge overtakes it by $n = 200$ (2852 against 2909) and is cheapest at $n = 500$ (8690 against 16,544), in line with the run-based analysis of [5].

TABLE IV

MEAN $\pm$ STANDARD DEVIATION AT $n = 100$ OVER 1000 PAIRED TRIALS. $W = C + M$; R IN UNITS OF 10^{-4} . REVERSE-SORTED IS DETERMINISTIC (ONE TRIAL; $\dagger$ ONE RANDOM-PIVOT DRAW). R IS UNDEFINED FOR FEW-UNIQUE INPUT.

Algorithm	Uniform-random		Nearly-sorted		Reverse-sorted		Few-unique
	W	R	W	R	W	R	W
Bubble	$12,321 \pm 549.4$	0.406 ± 0.011	$5,050 \pm 908.8$	0.118 ± 0.022	19,800	0.505	$10,621 \pm 539.7$
Insertion	$5,154 \pm 349.8$	0.972 ± 0.003	801 ± 191.8	0.745 ± 0.072	9,999	1.000	$4,154 \pm 332.4$
Merge	$1,886 \pm 5.8$	2.657 ± 0.189	$1,798 \pm 21.9$	0.338 ± 0.105	1,660	6.024	$1,868 \pm 8.9$
Heap	$2,772 \pm 31.2$	1.808 ± 0.139	$2,978 \pm 14.8$	0.205 ± 0.066	2,492	4.013	$2,378 \pm 70.0$
Quick (last pivot)	$1,486 \pm 165.9$	3.412 ± 0.438	$2,654 \pm 623.7$	0.245 ± 0.104	5,100	1.961	$1,607 \pm 116.4$
Quick (median-of-3)	$1,590 \pm 115.5$	3.167 ± 0.309	$1,162 \pm 173.5$	0.524 ± 0.154	2,174	4.600	$2,168 \pm 128.3$
Quick (random pivot)	$1,645 \pm 164.3$	3.074 ± 0.360	$1,159 \pm 115.6$	0.524 ± 0.157	1,710 †	5.848	$1,853 \pm 110.4$
Natural merge	$1,768 \pm 27.0$	2.835 ± 0.205	$1,119 \pm 85.9$	0.543 ± 0.163	1,791	5.583	$1,715 \pm 23.2$

TABLE V

MEAN COST $W = C + M$ ON UNIFORM-RANDOM INPUT FOR INCREASING n (100 TRIALS FOR $n \leq 500$, 30 FOR $n \geq 1000$).

Algorithm	$n = 10$	20	50	100	200	500	1000	2000
Bubble	110	466	3,059	12,264	49,463	312,448	1,247,663	4,989,747
Insertion	62	226	1,339	5,117	20,201	126,463	501,237	1,999,073
Merge	91	239	794	1,886	4,370	12,832	28,669	63,309
Heap	119	330	1,139	2,769	6,538	19,547	44,108	98,208
Quick (last pivot)	53	160	587	1,476	3,598	11,328	26,264	57,525
Quick (median-of-3)	72	192	645	1,574	3,768	11,378	25,578	57,478
Quick (random pivot)	67	187	659	1,633	3,903	12,180	27,895	62,230
Natural merge	79	219	728	1,768	4,153	12,353	27,279	60,578

TABLE VI

EXACT CANONICAL ENSEMBLE AT $n = 100$: FOR EACH β , THE MEAN AND MODAL DISORDER, THE CORRESPONDING β_{eff} , AND THE FREE ENERGY $F = -\beta^{-1} \ln Z(\beta)$.

β	$\langle E \rangle_\beta$	mode m	β_{eff} at mode	$F(\beta)$
-20	0.523	2590	-20.01	18.698
-10	0.511	2533	-10.08	36.880
0	0.500	2475	0.00	—
10	0.489	2417	10.08	-35.880
20	0.477	2360	20.01	-17.698
50	0.443	2190	50.03	-6.803
100	0.389	1922	99.99	-3.194
200	0.300	1475	200.10	-1.426
400	0.190	934	400.19	-0.594

TABLE VII

EFFECTIVE INVERSE TEMPERATURE ALONG TRAJECTORIES AT $n = 100$ (30 UNIFORM-RANDOM INPUTS). β_{eff} AT THE MOST NEGATIVE RECORDED STATE PER TRIAL, THEN AVERAGED.

Algorithm	Neg.- β steps (%)	Mean most-neg. β_{eff}	Min. β_{eff}
Bubble	3.1	-3.89	-50.56
Insertion	12.8	-3.89	-50.56
Merge	17.2	-3.89	-50.56
Heap	33.3	-343.23	-414.61
Quick (last pivot)	2.9	-3.89	-50.56

G. Effective inverse temperature along trajectories

Table VI checks the canonical ensemble of Section II-D against the exact landscape at $n = 100$: as β increases from -20 to 400, the modal m moves monotonically down from 2590 to 934, $\langle E \rangle_\beta$ tracks it, and β_{eff} evaluated at the mode recovers the input β to within rounding, confirming (6) as the corresponding local inverse-temperature relation. $\beta = 0$ reproduces the uniform distribution exactly ($\langle E \rangle_0 = 0.500$, mode $m = 2475$).

We then evaluate β_{eff} along the 30 uniform-random trajectories of Section VI-C at $n = 100$. Because Bubble, Insertion, Merge and Quick Sort never increase m (Section IV), the most extreme macrostate any of them visits is m_0 itself, so the most negative β_{eff} on each of their trajectories occurs at m_0 : a mean $\beta_{\text{eff}}(m_0) = -3.89$ over the 30 inputs, close to zero because E_0 is itself close to the entropy peak. Heap Sort reaches a substantially more negative inverse-temperature regime: its

most negative recorded state has a mean $\beta_{\text{eff}} = -343.2$ (minimum -414.6 over the 30 inputs), nearly two orders of magnitude larger in magnitude than the monotone algorithms' mean, and 33.3% of its recorded states lie at negative β_{eff} against 2.9–17.2% for the other four (Table VII). This is the same asymmetry Section VI-C reports as Λ , now expressed on the ensemble's inverse-temperature scale rather than as a path length: Heap Sort's heap-construction phase pushes the array toward low-multiplicity, high-disorder macrostates on the far side of the entropy peak.

VII. DISCUSSION

A. What the framework adds

Because $E_f = S_f = 0$ for every complete sort, endpoint quantities are identical across algorithms and $R = E_0/W$ carries no more ranking information than W (Proposition 1). What the framework adds is the path: the exact step-by-step description of Section IV, the excursion and drop-size statistics, the entropy path length Λ , and the effective inverse temperature β_{eff} of Section VI-G, all functions of $S(m)$ rather than of m or E alone. Bubble, Insertion, Merge and Quick

Sort all move monotonically downward in m , at costs that differ by a factor of eight, while Heap Sort's $\Lambda = 3.28$ and $\beta_{\text{eff}} \approx -343$ at its most negative point record a genuine excursion into a much lower-multiplicity region of permutation space. The distinction is not quantified by m or E alone: a peak of $E \approx 0.79$ identifies the excursion in disorder but does not quantify how far it lies from the high-multiplicity centre of the entropy landscape. Comparing a positional macrostate with a structural one, such as run entropy [5], is a natural way to extend this.

B. Relation to physical energy

W is an abstract operation count, not energy. Studies that measure energy on devices, on smartphones [22], on Android [23], in Java [27], in meta-analysis [26] and across implementation languages and input patterns [32], or that treat energy-efficient sorting algorithmically [21], [29], provide the natural way to test whether W , with a weight on writes as in [19], predicts measured energy. We do not attempt that here. On the theory side, connecting a per-run entropy path to the stochastic thermodynamics of computation [28], [31] would require a physical model of the machine, and the classical bounds [9], [10], [18], [20], [25] remain background only.

C. Limitations and threats to validity

Each algorithm is a single implementation, and pivot rule and merge scheme matter, as the variants show. Costs are abstract counts in Python-level accounting, weighted equally; a swap is charged three writes by convention. The entropy analysis assumes distinct keys, so few-unique input is reported only through C , M and W . The trajectory results use 30 inputs at $n = 100$ on uniform-random input; they are fully explained by the structural propositions but not sampled across all distributions. The random-pivot result for reverse-sorted input is a single draw. Sizes stop at $n = 2000$, and the nearly-sorted generator produces one particular family of low-disorder inputs.

D. Future work

Two directions follow directly. First, other representations of inversion structure, including inversion sequences with pattern restrictions [24] and sorting operations acting on inversion sequences [30], suggest tracking the entropy of the inversion-sequence representation instead of m . Second, run-based macrostates that count descending as well as ascending runs [5] should remove the reverse-sorted penalty of the natural merge sort used here, and a sweep over the write weight λ in $W = C + \lambda M$ would test how sensitive the rankings are to the read/write asymmetry [19].

VIII. CONCLUSION

We described sorting runs as trajectories through an exactly computed entropy landscape on inversion-count macrostates. The step-level behaviour of each algorithm is characterised exactly. The $n = 100$ cost experiments use 10^3 paired trials, while the trajectory experiment uses 30 uniform-random

inputs; in the latter, Heap Sort raises disorder in 83.0% of its steps and traverses 3.3 times its net entropy change. Cost rankings depend on input shape and size, with Insertion Sort overtaking Merge Sort at $n \approx 23$ and last-pivot Quick Sort moving from cheapest to third on nearly-sorted and reverse-sorted input. The disorder reduction rate equals E_0/W , so it normalises rather than ranks; the entropy path is what the framework adds. The framework is abstract, and calibrating it against measured device energy is the main open step.

APPENDIX A REPRODUCIBILITY

All numbers and figures are generated by the supplementary code [35] (Python 3.12, NumPy 2.4, SciPy 1.17). `states.py` computes exact Mahonian counts with Python integers; `algorithms.py` holds the instrumented sorts; `fastcount.py` the count-only versions; `exp_main.py`, `exp_traj.py`, `exp_path.py` and `exp_ensemble.py` run the experiments; `gen_tables.py` and `figs.py` produce Tables IV–VII and the figures. The commands are `python exp_main.py main, fine, and sweep`, followed by `python exp_traj.py, exp_path.py, exp_ensemble.py, gen_tables.py and figs.py`. The seed entropy is 20260927 throughout. The entire pipeline, including the exact $n = 100$ Mahonian table (under 0.1 s) and the canonical-ensemble check of Table VI, runs in a few minutes on one CPU core.

REFERENCES

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [2] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [3] R. M. Gray, *Entropy and Information Theory*, 2nd ed. Springer, 2011. [Online]. Available: <https://ee.stanford.edu/~gray/it.pdf>
- [4] J. Kahn and J. H. Kim, "Entropy and sorting," *J. Comput. Syst. Sci.*, vol. 51, no. 3, pp. 390–399, 1995.
- [5] O. Gila, M. T. Goodrich, and V. Sridhar, "How to sort in a refrigerator: Simple entropy-sensitive strictly in-place sorting algorithms," in *Proc. LATIN 2026*; arXiv:2603.05676 [cs.DS], 2026.
- [6] M. P. Schellekens, "Entropy conservation for comparison-based algorithms," *Topology Appl.*, vol. 309, Art. no. 107913, 2022.
- [7] B. List, M. Maucher, U. Schöning, and R. Schuler, "Randomized Quicksort and the entropy of the random source," in *Computing and Combinatorics (COCOON 2005)*, LNCS vol. 3595. Springer, 2005, pp. 450–460.
- [8] MIT 6.006, *Introduction to Algorithms*, "Lecture 10: Sorting III: Linear bounds/linear-time sorting," MIT OpenCourseWare, 2008. [Online]. Available: <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/>
- [9] R. Landauer, "Irreversibility and heat generation in the computing process," *IBM J. Res. Develop.*, vol. 5, no. 3, pp. 183–191, 1961.
- [10] C. H. Bennett, "Logical reversibility of computation," *IBM J. Res. Develop.*, vol. 17, no. 6, pp. 525–532, 1973.
- [11] J. C. Baez and M. Stay, "Algorithmic thermodynamics," *Math. Struct. Comput. Sci.*, vol. 22, no. 5, pp. 771–787, 2012.
- [12] M. Mézard and A. Montanari, *Information, Physics, and Computation*. Oxford Univ. Press, 2009.
- [13] P. A. MacMahon, *Combinatory Analysis*, vol. 1. Cambridge Univ. Press, 1915.
- [14] R. P. Stanley, *Enumerative Combinatorics, Vol. 1*, 2nd ed. Cambridge Univ. Press, 2012.
- [15] M. Bóna, *Combinatorics of Permutations*, 2nd ed. CRC Press, 2012.
- [16] B. H. Margolius, "Permutations with inversions," *J. Integer Sequences*, vol. 4, Art. 01.2.4, 2001.

- [17] E. Ben-Naim, “On the mixing of diffusing particles,” arXiv:1010.2563, 2010.
- [18] P. Faist, F. Dupuis, J. Oppenheim, and R. Renner, “The minimal work cost of information processing,” *Nature Commun.*, vol. 6, Art. no. 7669, 2015, doi:10.1038/ncomms8669.
- [19] G. E. Blelloch, J. T. Fineman, P. B. Gibbons, Y. Gu, and J. Shun, “Sorting with asymmetric read and write costs,” in *Proc. 27th ACM Symp. Parallelism Algorithms Architectures (SPAA)*, 2015, pp. 1–12, doi:10.1145/2755573.2755604.
- [20] A. S. Wright, “The physics of forgetting: Thermodynamics of information at IBM 1959–1982,” *Perspectives Sci.*, vol. 24, no. 1, pp. 112–141, 2016, doi:10.1162/POSC_a_00194.
- [21] B. Geissmann and P. Penna, “Sorting processes with energy-constrained comparisons,” *Phys. Rev. E*, vol. 97, no. 5, Art. no. 052108, 2018, doi:10.1103/PhysRevE.97.052108.
- [22] M. Verma and K. R. Chowdhary, “Analysis of energy consumption of sorting algorithms on smartphones,” *SSRN Electron. J.*, 2018, doi:10.2139/ssrn.3168550.
- [23] A. Schuler and G. Anderst-Kotsis, “Examining the energy impact of sorting algorithms on Android: An empirical study,” in *Proc. 16th EAI Int. Conf. Mobile Ubiquitous Syst. (MobiQuitous)*, 2019, pp. 404–413, doi:10.1145/3360774.3360808.
- [24] J. S. Kim and Z. Lin, “Pattern-avoiding inversion sequences and open partition diagrams,” *Theor. Comput. Sci.*, vol. 841, pp. 186–197, 2020, doi:10.1016/j.tcs.2020.07.011.
- [25] I. Georgescu, “60 years of Landauer’s principle,” *Nature Rev. Phys.*, vol. 3, p. 770, 2021, doi:10.1038/s42254-021-00400-8.
- [26] G. Dlamini, F. Jolha, Z. Kholmatova, and G. Succi, “Meta-analytical comparison of energy consumed by two sorting algorithms,” *Inf. Sci.*, vol. 582, pp. 767–777, 2022, doi:10.1016/j.ins.2021.09.061.
- [27] K. Carter, S. M. G. Ho, M. M. A. Larsen, M. Sundman, and M. H. Kirkeby, “Energy and time complexity for sorting algorithms in Java,” arXiv:2311.07298, 2023.
- [28] G. Manzano, G. Kardeş, É. Roldán, and D. H. Wolpert, “Thermodynamics of computations with absolute irreversibility, unidirectional transitions, and stochastic computation times,” *Phys. Rev. X*, vol. 14, Art. no. 021026, 2024, doi:10.1103/PhysRevX.14.021026.
- [29] V. Jayapaul, S. Jo, K. V. Palem, and S. R. Satti, “Energy efficient sorting, selection and searching,” *Theor. Comput. Sci.*, vol. 1002, Art. no. 114592, 2024, doi:10.1016/j.tcs.2024.114592.
- [30] T. Mansour, H. Skogman, and R. Smith, “Sorting inversion sequences,” *Discrete Math. Theor. Comput. Sci.*, vol. 27, no. 1, 2025, doi:10.46298/dmtcs.14010.
- [31] P. Chattopadhyay, A. Misra, T. Pandit, and G. Paul, “Landauer principle and thermodynamics of computation,” *Rep. Prog. Phys.*, vol. 88, no. 8, Art. no. 086001, 2025, doi:10.1088/1361-6633/add6b3.
- [32] M. S. Islam *et al.*, “Cross-language evaluation of sorting algorithms on various input patterns: A comparative study,” *J. Comput. Lang.*, vol. 87, Art. no. 101397, 2026, doi:10.1016/j.cola.2026.101397.
- [33] H. Mannila, “Measures of presortedness and optimal sorting algorithms,” *IEEE Trans. Comput.*, vol. C-34, no. 4, pp. 318–325, 1985.
- [34] V. Estivill-Castro and D. Wood, “A survey of adaptive sorting algorithms,” *ACM Comput. Surv.*, vol. 24, no. 4, pp. 441–476, 1992.
- [35] P. Mishra, “Supplementary code and data for ‘Entropy landscapes and kinetics of classical sorting algorithms,’” 2026. [Online]. Available: <https://github.com/punyajitmishra/entropy-sorting-landscapes>